

TokTickIT อธิบายระดับ Low-Level (CS Deep Dive)

เอกสารเล่มแรกอธิบายว่า "แต่ละส่วนทำอะไร" เล่มนี้อธิบายว่า "มันทำงานยังไงจริงๆ ในระดับเครื่อง" และที่สำคัญกว่านั้นคือ "ถ้ามันพัง ต้องดูตรงไหน แยกยังไง" — ทุกหัวข้อจะปิดท้ายด้วย **Debug Playbook** ที่อิงจากปัญหาจริงที่เจอระหว่างทำ lab นี้

บทที่ 1 — Node.js Event Loop และทำไม Express ถึงทำงานแบบนี้

1.1 Node.js เป็น single-threaded — แปลว่าอะไรจริงๆ

JavaScript ใน Node รันบน **thread เดียว** (ยกเว้นงานเบื้องหลังบางอย่างที่ libuv โยนไป thread pool เช่น file I/O, DNS lookup) ทุกบรรทัด synchronous code รันเรียงกันห้ามแทรก ไม่มี race condition แบบ multi-thread ทั่วไป (ไม่ต้องใช้ mutex/lock) แต่แลกมาด้วยข้อจำกัด: ถ้าบรรทัดไหน **block** นานๆ ทั้งเซิร์ฟเวอร์หยุดรับ request ใหม่

Event loop คือ วงจรที่คอยเช็ค ว่า "มีงานที่ทำเสร็จแล้วรอส่งผลลัพธ์กลับมาไหม" วนไปเรื่อยๆ เป็นเฟส: timers → pending callbacks → poll (I/O) → check (setImmediate) → close callbacks

1.2 async / await คือ syntax sugar ของอะไร

```
app.get("/api/categories", async (_req, res) => {
  const categories = await prisma.category.findMany({ orderBy: { id: "asc" } });
  res.json(categories);
});
```

async function คำนวณเป็น **Promise** เสมอ และ await คือจุดที่บอก JS engine ว่า "คืน control กลับไปให้ event loop ก่อน แล้วค่อยกลับมาทำต่อบรรทัดนี้เมื่อ Promise resolve" มันไม่ใช่การ "รอแบบ block thread" — ระหว่างรอ query จาก Postgres, Node ยังรับ request อื่นได้ปกติ เบื้องหลังคือ .then() chain ที่ compiler แปลงให้ (await x ≈ x.then(value => { ต่อจากตรงนี้ })) เก็บ state ของฟังก์ชันไว้ใน closure ที่เรียกว่า **continuation**

1.3 ทำไม createApp() ไม่ block

prisma.category.findMany() ไปสุดท้ายเรียก native driver (pg) ซึ่งเปิด TCP socket คุยกับ Postgres การเขียน/อ่าน socket เป็นงาน I/O ที่ libuv จัดการแบบ non-blocking (ใช้ epoll บน Linux / IOCP บน Windows) Node แค "ฝากคำสั่ง" ไว้ แล้วกลับไปทำ request อื่นทันที พอ Postgres ตอบกลับมา (ผ่าน socket event) event loop ถึงจะเรียก callback (resume ฟังก์ชัน async ที่ await ค้างอยู่)

Debug Playbook — Event Loop / Async

อาการ	สาเหตุที่เป็นไปได้	วิธีเช็ค/แก้
Server "ค้าง" ไม่ตอบ request ไหนเลย	มี synchronous loop หนักๆ block thread (เช่น sort array ใหญ่มาก, regex catastrophic backtracking)	ใช้ node --prof หรือ Chrome DevTools profiler เช็ค ว่า call stack ไหนกินเวลานาน
UnhandledPromiseRejectionWarning ใน console	await ถูกเรียกใน context ที่ไม่มี try/catch หรือ error หลุดไปไม่มีใครจับ	ห่อ route handler ด้วย try/catch หรือใช้ error-handling middleware ของ Express
Response ค้างไม่กลับมาเลย (ไม่ error, ไม่ timeout)	ลืมเรียก res.json()/res.send() ใน path ใด path หนึ่งของ logic (เช่น if ที่ไม่มี else แล้วลืมตอบ)	เช็คทุก branch ของ route handler ว่ามี response ครบทุกทาง

บทที่ 2 — CORS: กลไกจริงของ Browser ไม่ใช่แค่ "config"

2.1 Same-Origin Policy คืออะไรในระดับ browser engine

Browser เปรียบเทียบ **origin** ของหน้าเว็บ (protocol + host + port) กับ origin ปลายทางที่ request ไป ถ้าไม่ตรงกัน ถือเป็น **cross-origin request** ซึ่งโดน sandbox กันโดย browser's network stack เอง (ไม่ใช่ server เป็นคนบล็อก — request มันถูกส่งออกไปจริง แต่ browser ปฏิเสธไม่ยอมให้ JavaScript อ่าน response กลับมา ถ้า server ไม่ตอบ header อนุญาต)

2.2 Simple Request vs Preflighted Request

- **Simple request:** GET/POST/HEAD ที่ header มาตรฐาน (ไม่มี custom header, content-type เป็น `text/plain`, `multipart/form-data`, หรือ `application/x-www-form-urlencoded`) → browser ส่ง request จริงไปเลย แล้วเช็ค response header `Access-Control-Allow-Origin` ที่หลัง
- **Preflighted request:** ถ้า request มี custom header หรือ `Content-Type: application/json` (แบบที่ frontend เราใช้) → browser จะยิง **OPTIONS request** ไปก่อน ถามว่า "ผมจะส่ง GET/POST พร้อม header พวกนี้ อนุญาตไหม" ต้องได้คำตอบ `Access-Control-Allow-Origin` ที่ตรงกับ origin ของหน้าเว็บก่อน ถึงจะยอมส่ง request จริงตามไป

โค้ดใน `app.ts` :

```
app.use(cors({ origin: allowedOrigins() }));
```

middleware `cors()` คือคนที่ดักจับ **OPTIONS request** preflight นี้ แล้วตอบ header ที่ถูกต้องให้อัตโนมัติ ถ้าไม่มี middleware นี้ Express จะไม่รู้จักร `OPTIONS` method เลย (ไม่มี route ไหนดักไว้) ผลคือ browser preflight fail แล้วก็ block request จริงไม่ให้ออกไปเลย (บาง browser ยิ่งจริงแต่ block ตอนอ่าน response — พฤติกรรมนี้ต่างกันไปตาม browser)

2.3 วิธี debug CORS จริง (ไม่ใช่เดา)

1. เปิด DevTools → tab **Network**
2. หา request ที่ fail แล้วดูว่ามี request แถวนสุดชื่อ `OPTIONS` ไปยัง URL เดียวกันไหม (preflight)
3. ถ้า `OPTIONS` ตอบ 200 แต่ request จริง (GET/POST) ยัง fail → ปัญหาไม่ใช่ CORS แล้ว (เช็ค server error)
4. ถ้า `OPTIONS` ไม่ตอบเลยหรือตอบ error → เช็คค่า origin ของหน้าเว็บ (ดูใน address bar) ตรงกับค่าที่ ตั้งใน `CLIENT_ORIGIN` เป๊ะๆ ทั้ง protocol, host, และ **port** (คนละ port = คนละ origin แม้ host เดียวกัน)
5. Error message ใน Console tab ที่ขึ้นต้นด้วย `Access to fetch at ... has been blocked by CORS policy` จะบอก origin ที่ browser คาดหวังจริงๆ — เอาไปเทียบกับ `.env` ได้เลย

นี่คือ **root cause** ของบั๊ก "still a blank site" ที่เจอในโปรเจกต์นี้: React dev server รันที่ port ที่ไม่ตรงกับ

`CLIENT_ORIGIN` เพราะ port เริ่มต้น (5173) ถูกใช้ไปแล้วโดย process อื่น Vite เลย fallback ไป port ถัดไป (5174) โดยอัตโนมัติ แต่ `.env` ยังชี้ไปที่ 5173 เดิม

บทที่ 3 — Prisma/PostgreSQL: สิ่งที่เกิดขึ้นจริงใต้ ORM

3.1 Connection Pool คืออะไรจริงๆ

การเปิด TCP connection ไปหา Postgres แต่ละครั้งมี cost สูง: three-way handshake (SYN, SYN-ACK, ACK), แล้ว Postgres ต้อง fork process ใหม่ (Postgres ใช้ process-per-connection ไม่ใช่ thread-per-connection) เพื่อจัดการ connection นั้น ถ้าเปิดใหม่ทุก request จะช้ามากและกิน memory ฝั่ง Postgres เร็ว

Connection pool คือชุด connection ที่เปิดค้างไว้ล่วงหน้า (เช่น 10 เส้น) แล้วแจกยืมให้แต่ละ query ใช้ชั่วคราวแล้วคืนกลับเข้า pool (ไม่ปิดจริง) — เหมือนแท็กซี่ที่จอดรอเป็นคิว ไม่ต้องเรียกรถใหม่ทุกเที่ยว

```
export const prisma = new PrismaClient({ adapter });
```

`PrismaClient` แต่ละ instance ผูกกับ pool ของตัวเอง (ผ่าน `PrismaPg` adapter ที่ wrap `pg.Pool` จาก `node-postgres`) นี่คือเหตุผลที่คอมเมนต์ในโค้ดเตือนว่า "สร้าง instance ใหม่ทุก request จะเปิด connection ท่วมจนเบียดโควตาของ Postgres" — Postgres มี `max_connections` จำกัด (default 100) ถ้า pool ใหม่ถูกสร้างซ้ำๆ ไม่มีที่สิ้นสุดสุดท้าย connection จะเต็มโควตาและ query ใหม่จะได้ error `too many clients already`

3.2 Migration ทำงานยังไงจริงๆ

`npx prisma migrate dev` ไม่ได้แคร์ SQL ตรงๆ มันมีขั้นตอน:

1. สร้าง **shadow database**ชั่วคราว (database คู่ขนานที่ว่างเปล่า)
2. รัน migration ทั้งหมดที่มีอยู่ใน `prisma/migrations/` ไปตามลำดับใน shadow database
3. เปรียบ schema ที่ได้จาก shadow database กับ `schema.prisma` ปัจจุบัน — ถ้าไม่ตรงกัน generate migration ใหม่ (diff)
4. รัน migration ใหม่ใน database จริง แล้วบันทึกไว้ในตาราง `_prisma_migrations` (ตารางเมทาเดตาที่ Prisma สร้างเองเพื่อ track ว่า migration ไหนรันไปแล้วบ้าง)

เหตุผลที่ต้องมี shadow database: เพื่อตรวจจับ **schema drift** — กรณีที่มีคนไปแก้ตารางตรงๆ ผ่าน SQL client โดยไม่ผ่าน migration file ทำให้ database จริงกับประวัติ migration ไม่ตรงกัน Prisma จะ error ทันทีแทนที่จะรัน migration ผิดๆ ทั้ง

3.3 N+1 Query Problem (ทำไมต้องรู้แม่โปรเจกต์นี้ยังไม่เจอ)

ถ้าวันหนึ่งต้องดึง Ticket พร้อม PublicComments ของแต่ละใบ โค้ดแบบไร้เดียงสาจะเป็น:

```
const tickets = await prisma.ticket.findMany();
for (const t of tickets) {
  t.comments = await prisma.comment.findMany({ where: { ticketId: t.id } }); // query แยกทุกใบ!
}
```

ถ้ามี 100 tickets จะยิง query ไป Postgres **101 ครั้ง** (1 + N) แทนที่จะเป็นครั้งเดียว วิธีแก้คือใช้ `include` :

```
const tickets = await prisma.ticket.findMany({ include: { comments: true } });
```

Prisma จะแปลงเป็น SQL `JOIN` เดียว (หรือ batch query ที่ฉลาดกว่า ขึ้นกับ relation type) ลดจาก 101 round-trip เหลือ 1

Debug Playbook — Database Layer

อาการ	สาเหตุ	วิธีแก้
<code>PrismaClientInitializationError</code> ตอนสร้าง client	Prisma 7 ไม่อ่าน <code>DATABASE_URL</code> เองแล้ว ต้องส่ง adapter ตรงๆ	ตรวจว่ามี <code>new PrismaPg({ connectionString })</code> ส่งเข้า <code>new PrismaClient({ adapter })</code>
<code>Can't reach database server</code>	Postgres container ไม่ได้รันอยู่ หรือ port/host ผิด	<code>docker ps</code> เช็คดูว่า container ขึ้นอยู่ไหม, ตรวจ <code>DATABASE_URL</code> ว่า host/port ตรงกับ <code>docker-compose.yml</code>

Unique constraint failed on the fields: (name)	มีการ create() ซ้ำชื่อที่ @unique ไว้แล้ว (เช่นรัน seed ด้วย create แทน upsert)	เปลี่ยนเป็น upsert หรือเช็คว่ามีอยู่แล้วหรือยัง
too many clients already	สร้าง PrismaClient ใหม่ซ้ำๆ โดยไม่ปิด pool เดิม (เช่นสร้างใน loop หรือทุก request)	ใช้ instance เดียว (singleton) แพร่ทั้งแอป ตามที่ prisma.ts ทำ
Query ซ้ำติดปกติเมื่อข้อมูลเยอะขึ้น	อาจเป็น N+1, หรือ column ที่ query บ่อยไม่มี index	รัน EXPLAIN ANALYZE ใน psql ดู query plan, พิจารณาเพิ่ม @@index ใน schema

บทที่ 4 — Testing: กลไกภายใน ไม่ใช่แค่ syntax

4.1 vi.spyOn ทำงานยังไงจริงๆ (Monkey Patching)

```
vi.spyOn(globalThis, 'fetch').mockResolvedValueOnce(Response.json({...}))
```

เบื้องหลังคือ **property replacement**: Vitest จัด reference เดิมของ `globalThis.fetch` ไว้แล้ว เขียนทับ property `fetch` ด้วยฟังก์ชันปลอมของมันเอง (นี่คือเทคนิคที่เรียกว่า monkey patching) เมื่อโค้ดจริงเรียก `fetch(...)` มันไม่รู้ตัวเลยว่ากำลังเรียกของปลอม เพราะมันดู reference จาก `globalThis.fetch` เหมือนเดิม — key insight: การ **mock** ได้ ต้องมี **object** ที่ **mutable** และเข้าถึง **property** ได้จากภายนอก ถ้าโค้ดเดิม `import fetch` แบบ `import { fetch } from 'some-lib'` แล้ว bind ค่าไว้ตรงๆ (ไม่ผ่าน `globalThis`) จะ mock ไม่ได้ด้วยวิธีนี้ ต้องใช้ `vi.mock()` ระดับ module แทน `mockResolvedValueOnce` แปลว่า "ครั้งถัดไปที่ถูกเรียก ให้ resolve ด้วยค่านี้ครั้งเดียว แล้วคืนพฤติกรรม เดิม (หรือ mock ตัวถัดไปที่ queue ไว้)" — ระบบ mock เก็บ queue ภายในเป็น FIFO เรียงตามลำดับที่ `mockResolvedValueOnce` ถูกเรียก ตรงกับลำดับการเรียก `fetch` จริงในโค้ด (health check ก่อน, categories ที่หลัง)

4.2 jsdom คืออะไร มันไม่ใช่ browser จริง

`jsdom` คือ implementation ของ DOM API เขียนด้วย pure JavaScript รันบน Node — มันจำลอง `document`, `window`, event system ได้ประมาณ 90% ของ browser จริง แต่ **ไม่มี layout engine จริง** (ไม่คำนวณ CSS layout จริง, ไม่มี rendering pipeline, ไม่รองรับบาง Web API เช่น `IntersectionObserver` เต็มรูปแบบ) เพราะฉะนั้น test ที่พึ่งพาขนาดจริงของ element บนจอ (เช่น `getBoundingClientRect()`) จะได้ค่า 0 เสมอใน `jsdom` — ถ้าต้องทดสอบสิ่งเหล่านี้จริงๆ ต้องใช้ browser จริงผ่าน `Playwright/Cypress (E2E test)` ไม่ใช่ `jsdom (unit/integration test)`

4.3 ทำไม waitFor จำเป็น — Race Condition ใน Test

```
await user.click(screen.getByRole('button', { name: /Check System/i }));
await waitFor(() => {
  expect(screen.getByText('Hardware')).toBeInTheDocument()
})
```

`user.click()` แต่ trigger event ให้ React เริ่มทำงาน (เรียก `handleCheckSystem` ซึ่งเป็น `async`) แต่ React re-render **ไม่ได้เกิดขึ้นทันทีแบบ synchronous** — มันรอ microtask queue ว่าง (หลัง `await fetchHealth()` resolve) ก่อนจะ schedule re-render ถ้า assert ทันทีหลัง `click()` โดยไม่มี `waitFor` จะเจอ DOM ที่ยังเป็น **state เก่า** (`idle` หรือ `loading`) ทำให้ test fail แบบ flaky (บางทีผ่านบางทีไม่ผ่าน ขึ้นกับ timing ของเครื่อง) `waitFor` แก้ปัญหานี้ด้วยการ **poll** ซ้ำ (เช็คทุก ไม่กี่ ms จนกว่า assertion จะผ่านหรือ timeout) แทนที่จะเช็คครั้งเดียว

Debug Playbook — Testing

อาการ	สาเหตุ	วิธีแก้
Test ผ่านบางครั้ง ไม่ผ่านบางครั้ง (flaky)	ขาด await/waitFor รอบ assertion ที่เช็คผลลัพธ์ของ async operation	ห่อ assertion ด้วย waitFor(() => {...}) หรือใช้ findBy* แทน getBy* (ตัวมันมี wait ในตัว)
TestingLibraryElementError: Unable to find element	Text ที่หาไม่ตรงกับที่ render จริง (พิมพ์ผิด, case ไม่ตรง, หรือ element ยังไม่ render เพราะ state ผิด phase)	screen.debug() เพื่อ print DOM จริงที่ render ออกมา เปรียบเทียบกับที่คาดหวัง
Mock ทำงานไม่ตรงลำดับที่คาดหวัง	mockResolvedValueOnce ถูกเรียกผิดลำดับเทียบกับที่โค้ดจริงเรียก fetch	ไล่อ่านโค้ดจริงว่าเรียก fetch ก็ครั้งเรียงอะไรก่อนหลัง แล้วเรียง mock ให้ตรง
Test ผ่านตอนรันเดี่ยว แต่ fail ตอนรันพร้อมกันทั้งไฟล์	Mock ตัวก่อนหน้าไม่ได้ถูก reset ทำให้ state รั่วข้าม test	เช็คว่ามี vi.restoreAllMocks() ใน beforeEach/afterEach

บทที่ 5 — React: กลไกภายใน Reconciliation

5.1 Virtual DOM คืออะไรจริงๆ

ทุกครั้งที่ component render ใหม่ React ไม่ได้เขียนลง DOM จริงทันที มันสร้าง **object tree ใน memory** (virtual DOM) ที่อธิบายว่าหน้าจออนาคตมีหน้าตาอย่างไรก่อน แล้วเทียบ (diff) กับ tree ของรอบก่อนหน้า (reconciliation algorithm) — เจาะจุดต่างตรงไหน ถึงจะสั่งแก้ เฉพาะจุดนั้น ใน DOM จริง (ซึ่งการแก้ DOM จริงเป็นปฏิบัติการที่ expensive กว่าเทียบ object ใน memory มาก — นี่คือเหตุผลที่ React เร็วกว่าการ เขียน `innerHTML` ใหม่ทั้งหน้าทุกครั้ง)

`key` prop (เช่น `key={category.id}` ใน `.map()`) คือสิ่งที่บอก React ว่า **element ไหนคือ element เดิมที่ย้ายตำแหน่ง** vs **element ใหม่ที่เพิ่งเกิด** ถ้าไม่มี `key` หรือใช้ `index` เป็น `key` ตอน list เปลี่ยนลำดับ React จะ diff ผิดพลาด (คิดว่า element เดิมเปลี่ยนเนื้อหา ทั้งที่จริงมันคือ คนละ element ที่ย้ายตำแหน่ง) ทำให้เกิด bug UI สลับตำแหน่งข้อมูลผิดหรือ state รั่วข้าม row

5.2 State Update เป็น Asynchronous และ Batched

```
setCheck({ phase: 'loading' })
await fetchHealth()
```

`setCheck(...)` ไม่ได้อัปเดต `check` ทันทีที่บรรทัดนี้รัน — React เก็บคำขอเปลี่ยน state ไว้ใน queue แล้ว process ที่เดียวตอนจบ event handler (เรียกว่า **batching**) เพื่อลดจำนวนรอบ re-render (ถ้าเรียก `setState` สามครั้งติดกันใน handler เดียว React รวมเป็น re-render ครั้งเดียว ไม่ใช่ 3 ครั้ง)

ผลที่ตามมา: ถ้าอ่านค่า `check` ทันทีที่บรรทัดถัดจาก `setCheck(...)` จะยังเห็นค่าเก่า (นี่คือกับดักคลาสสิก ของมือใหม่ React) เพราะ `check` เป็นตัวแปรจาก closure ของ render รอบปัจจุบัน ไม่ใช่ reference ที่ อัปเดต mutable ได้ตรงๆ — ค่าจริงจะอัปเดตก็ต่อเมื่อ component **render รอบถัดไป**

5.3 Closure และ Stale State (กับดักที่พบบ่อยที่สุด)

ทุกครั้งที่ component function รันใหม่ (re-render) มันสร้าง **scope ใหม่ทั้งหมด** รวมถึงตัวแปร, ฟังก์ชันข้างในทุกตัว การอ้างอิงตัวแปร state จากใน callback ที่สร้างไว้ตอน render รอบเก่า จะ "ค้าง" (capture) ค่าของรอบนั้นไว้ตลอด ไม่เห็นค่าที่อัปเดตทีหลัง — นี่คือปัญหาที่เรียกว่า **stale closure** ตัวอย่างสมมติ (ไม่ได้อยู่ในโค้ดจริง แต่เป็น pattern ที่ต้องระวังเมื่อ codebase โตขึ้น):

```
useEffect(() => {
  const interval = setInterval(() => {
    console.log(check) // ค่านี้จะค้างเป็นค่าตอน effect รันครั้งแรกเสมอ ไม่ใช่ค่าปัจจุบัน
  }, 1000)
  return () => clearInterval(interval)
}, []) // dependency array ว่าง = effect รันครั้งเดียว ไม่รู้จัก check ตัวใหม่
```

Debug Playbook — React

อาการ	สาเหตุ	วิธีแก้
UI ไม่อัปเดตหลังกดปุ่ม ทั้งที่ logic ถูก	อ่านค่า state ทันทีหลัง setState (ยังไม่ re-render)	ใช้ useEffect ดัก state ใหม่ หรือส่งค่าที่คำนวณเองผ่านตัวแปร local แทนอ่านจาก state เก่า
List สลับตำแหน่งข้อมูลผิดตอนแก้ไข/เรียงใหม่	ใช้ index เป็น key แทน id จริง	เปลี่ยนไปใช้ unique id ที่ผูกกับข้อมูลจริง เช่น key={category.id}
ค่าที่ log/ใช้ใน setInterval/event listener เก่าค้าง ไม่อัปเดต	Stale closure จาก dependency array ที่ไม่ครบ	เติม dependency ที่ขาดใน useEffect, หรือใช้ useRef เก็บค่าล่าสุดถ้าตั้งใจไม่ยอม re-run effect
Component re-render บ่อยเกินจำเป็น (เช็คด้วย React DevTools Profiler)	สร้าง object/function ใหม่ทุก render แล้วส่งเป็น prop (reference เปลี่ยนทุกครั้งแม้ค่าเหมือนเดิม)	ใช้ useMemo/useCallback ห่อ ถ้า child component ทำ React.memo ไว้

บทที่ 6 — Git: Object Model จริงเบื้องหลังคำสั่ง

6.1 Git ไม่ได้เก็บ "diff" — มันเก็บ snapshot เดิม

ความเข้าใจผิดที่พบบ่อย: คนคิดว่า commit เก็บ "ส่วนต่าง" จากอันก่อนหน้า (เหมือน patch) แต่จริงๆ Git เก็บ **snapshot เดิมของทุกไฟล์ ณ ขณะนั้น** ในรูป object 3 ชนิด:

- blob — เนื้อหาไฟล์ดิบๆ (ไม่มีชื่อไฟล์ติดมาด้วย) hash ด้วย SHA-1/SHA-256 ของเนื้อหา ถ้าไฟล์สองไฟล์เนื้อหาเหมือนกันเป๊ะ (แม้คนละที่คนละชื่อ) จะใช้ blob **object เดียวกัน** (deduplication)
- tree — โครงสร้างโฟลเดอร์ ชี้ไปที่ blob (ไฟล์) หรือ tree อื่น (โฟลเดอร์ย่อย) พร้อมชื่อไฟล์
- commit — ชี้ไปที่ tree เดียว (root ของ snapshot ทั้งหมด) + ชี้ไปที่ parent commit(s)
 - metadata (author, message, timestamp)

git log ที่เห็นเป็นเส้นเรียงกันจริงๆ คือการเดิน **linked list ของ commit object** ที่แต่ละอันชี้ไป parent ของตัวเอง (ทำให้ list ย้อนกลับได้ แต่เดินไปข้างหน้าไม่ได้ตรงๆ — นี่คือเหตุผลที่ branch/tag มีไว้เพื่อ "จำ" ว่า commit ไหนคือปลายสุดปัจจุบัน)

6.2 git reset --hard ทำอะไรจริงๆ

```
git reset --hard 1af96a8
```

คำสั่งนี้ทำ 3 อย่างเรียงกัน:

- ย้าย **HEAD** (pointer ที่บอกว่า branch ปัจจุบันชี้ commit ไหน) ไปที่ 1af96a8
- ย้าย **branch ref** (เช่น feature/3-categories) ให้ชี้ commit เดียวกัน

3. เขียนทับ **working directory** ให้ตรงกับ snapshot ของ commit นั้น (`--hard` เท่านั้นที่ทำงานนี้ `--soft` หยุดแค่ข้อ 1-2, `--mixed` ทำถึงแค่ unstage แต่ไม่แตะไฟล์จริง)

commit ที่ไม่มี ref ไหนชี้ถึงแล้ว **ไม่ได้หายทันที** — มันยังอยู่ใน `.git/objects` จนกว่า garbage collector (`git gc`) จะมาเก็บกวาด (default รันอัตโนมัติเป็นระยะ หรือ manual) ระหว่างนั้น กู้คืนได้ผ่าน `git reflog` (log ของทุกจุดที่ HEAD เคยชี้ไปเก็บแยกจาก commit history ปกติ) — นี่คือเหตุผลที่ตอน reset ผิดพลาดในโปรเจกต์นี้ กู้คืนได้ทันทีด้วย `git reflog` แล้ว `git reset --hard` กลับไปที่ hash เดิม

6.3 ทำไม Force-Push อันตราย (Fast-Forward vs Non-Fast-Forward)

Remote ปกติจะยอมรับ push ก็ต่อเมื่อเป็น **fast-forward** (commit ใหม่ที่ push มา มี history เดิมของ remote อยู่ครบเป็น ancestor) ถ้า local กับ remote แตกกันไปคนละทาง (เช่นมีคนอื่น push ไปแล้วระหว่างที่เรา `reset --hard` ประวัติของตัวเอง) remote จะปฏิเสธ push ธรรมดา (`non-fast-forward`) `git push --force` คือการบอก remote **"ไม่ต้องเช็ค ให้เขียนทับ ref ไปเลย"** — ถ้ามีคนอื่น push commit ที่เรายังไม่เห็นไปแล้ว commit เหล่านั้นจะกลายเป็น **orphan** บน remote (ไม่มี ref ไหนชี้ถึง) และหายไปจากมุมมองปกติทันที (แม้จะยังกู้จาก reflog ของฝั่งเขาได้ในบางกรณี ถ้าเขายังไม่ prune)

`--force-with-lease` ปลอดภัยกว่า `--force` เพราะมันเช็คก่อนว่า **remote ยังอยู่ตรงจุดที่เราคิดว่ามันอยู่** ใหม่ (เทียบ ref ก่อน overwrite) ถ้ามีคน push แซงไปแล้วมันจะ reject แทนที่จะเขียนทับเจียบๆ

Debug Playbook — Git

อาการ	สาเหตุ	วิธีแก้
<code>reset --hard</code> ไป ผิด commit ข้อมูลหาย	ระบุ hash ผิด หรือ reset ก่อน commit งานที่ยังไม่ได้ save	<code>git reflog</code> หา HEAD ตัวก่อนหน้า แล้ว <code>git reset -hard <hash เดิม></code>
Force-push แล้ว เพื่อนบอกงานหาย	Local history ไม่มี commit ของเพื่อน เป็น ancestor ตอน force-push ทับ	ให้เพื่อนเช็ค reflog ฝั่งเขา (ถ้า fetch/pull commit นั้นมาแล้วก่อนหน้า) หรือกู้จาก local copy ของเพื่อนที่ยังมี commit นั้นอยู่
Merge conflict อ่านไม่เข้าใจ	ไม่รู้ว่่า <=====> แบ่งฝั่งไหนเป็นของใคร	ฝั่งบน (<<<<<< HEAD) คือโค้ดฝั่งเรา, ฝั่งล่าง (>>>>>> branch-name) คือโค้ดฝั่งที่ merge เข้ามา
Commit ไปผิด branch	ลืมเช็ค git branch ก่อน commit	<code>git log</code> หา hash ของ commit นั้น, <code>git cherry-pick <hash></code> ไปยัง branch ที่ถูก แล้ว <code>git reset --hard HEAD~1</code> เอาออกจาก branch ผิด

สรุป: วิธีคิดแบบ CS เวลาเจอบั๊กที่ไม่เคยเจอ

- แยกชั้น (layer) ที่ฟังก์ชัน** — เป็นปัญหาที่ browser (network tab), Express (server log), Prisma/Postgres (query error), หรือ React (component state)? อย่าเดา ให้ดู error message และ stack trace จริงว่าฟังก์ที่ layer ไหน
- เช็คว่า synchronous หรือ asynchronous** — ถ้าเกี่ยวกับ timing/race condition มักเป็นปัญหา async (ลืม await, ลืม waitFor, ลืม batching ของ React state)
- อ่าน error message ตัวแรกที่ขึ้น ไม่ใช่ตัวสุดท้าย** — error ที่ตามมาหลังจากนั้นมักเป็นผลพวง (cascading failure) ไม่ใช่ root cause จริง
- reproduce ให้ได้ก่อนแก้** — ถ้าแก้โดยไม่เข้าใจว่าทำไมพัง มีโอกาสสูงที่จะแก้ผิดจุดแล้วบั๊กโผล่ที่อื่นแทน